

Agent 可观测与追踪：Trace、指标与回放

语言策略：code (Java / Python / TypeScript)

TL;DR

- 普通服务监控只能告诉你“接口慢不慢”，Agent 可观测必须回答“它为什么这么走”。
- 最小可用能力是三项：**Trace 记录决策轨迹、指标聚合趋势、回放复现问题。**
- 可观测不是上线后补，而是在 Agent Loop 里埋点：LLM 调用、工具调用、状态变更、审批事件都必须留痕。

1. 与普通服务可观测的区别

维度	普通服务	Agent
一次请求	一次调用	多轮 LLM + 多轮工具
关键对象	接口延迟	轨迹、工具、上下文
错误定位	堆栈	哪一轮、哪个工具、哪段上下文
复现	重放请求	重放完整状态与上下文
成本	机器资源	Token 与工具费用

2. Trace 语义模型

Span 类型	记录内容
task	task_id、用户、租户、目标、最终状态
agent_loop	轮次、本轮思考摘要、停止原因
llm_call	model、prompt_version、输入输出 Token、延迟、采样参数
tool_call	工具名、参数摘要、返回摘要、风险等级、审批状态
memory	写入/读取的 memory key 与来源
retrieval	查询、命中数、rerank 前后顺序

一个 Span 最小属性：trace_id、task_id、parent_span_id、span_id、node、model、prompt_version、tool_name、status、latency_ms、input_tokens、output_tokens。

上下文必须脱敏后记录：密钥、手机号、邮箱、身份证号等先做 redaction，再写日志或导出。

3. 指标清单

指标	含义	告警建议
task_success_rate	任务成功率	低于 0.90 告警
avg_steps	平均 Agent 轮次	突增说明计划变差
tool_error_rate	工具调用错误率	超过 0.05 告警
approval_rate	高危动作审批率	突增要排查注入
tokens_per_task	单任务 Token	超预算告警
cost_per_task	单任务成本	超预算告警
p95_task_latency	任务 P95 延迟	超 SLO 告警
loop_exit_by_limit	因轮次上限退出的比例	持续大于 0.01 要修

4. 回放与调试

- 保存每个任务的输入、上下文快照、工具返回和最终输出；
- 按 task_id 重放，注入同一份上下文，观察模型是否做出不同决策；
- 回放数据必须脱敏并按保留周期清理；
- 生产事故复盘顺序：看最终输出 → 看工具轨迹 → 看上下文 → 重放验证。

5. TypeScript 版采集器

```
1 interface AgentSpan {
2   traceId: string;
3   taskId: string;
4   spanId: string;
5   parentSpanId?: string;
6   node: "task" | "agent_loop" | "llm_call" | "tool_call" | "memory"
7   | "retrieval";
8   model?: string;
9   promptVersion?: string;
10  toolName?: string;
11  status: "ok" | "error";
12  latencyMs: number;
13  inputTokens: number;
14  outputTokens: number;
15 }
16
17 class AgentTelemetry {
18   private spans: AgentSpan[] = [];
19
20   record(span: AgentSpan): void {
21     this.spans.push(span);
22   }
23
24   taskCount(): number {
25     return new Set(this.spans.map(s => s.taskId)).size;
26   }
27
28   toolErrorRate(): number {
29     const toolSpans = this.spans.filter(s => s.toolName);
30     if (toolSpans.length === 0) return 0;
31     const errors = toolSpans.filter(s => s.status === "error");
32     return errors.length / toolSpans.length;
33   }
34
35   totalTokens(): number {
36     return this.spans.reduce((sum, s) => sum + s.inputTokens + s.outputTokens, 0);
37   }
38 }
```

6. 典型故障排查表

现象	先查什么	常见原因
任务变慢	P95、平均步数、工具延迟	计划变长、工具超时、模型切换
成功率突降	失败任务轨迹与上下文	Prompt/工具描述变更、知识库污染
成本突增	单任务 Token、缓存命中率	上下文膨胀、路由到贵模型
工具报错多	tool_error_rate 按工具分列	Schema 变更、权限不足
循环退出多	loop_exit_by_limit	目标不清晰、工具返回不可用

7. 延伸阅读

- 大模型网关
- Agent 部署与发布
- Agent 评测体系